

1 INTRODUCTION

You have been hired to develop a pair of collaborating prototype systems: “eMART”  for handling online sales for a store and “eDEPOT”  for managing inventory of products in a warehouse to support eMART (and possibly other stores). The systems will manage, through their interactions, a product catalog for eMART and an inventory for eDEPOT, and sell products online. The main functionality includes the following tasks:

- Maintaining a catalog of products, customer and order information,
- Maintaining in the inventory system the information on individual products in the warehouse,
- Processing transactions (sales, orders, shipping, etc),
- Generating monthly reports, and
- Providing interfaces for online customers and store managers.

This document describes data and operational requirements as well as system requirements.

2 ONLINE STORE eMART

The online store maintains two key pieces of information: a product catalog and customer data (including their orders). The product catalog lists all products that may be sold to customers, while the customer data allows eMART to provide discounts to those who are “frequent” buyers.

2.1 Catalog

A catalog contains information about products (or items). The following information is stored for each item. (Unless specifically mentioned, you may assume that strings have at most 20 characters.)

Stock number

Each item has a unique stock number that has the form **XXnnnnn**, where **XX** is a two-letter combination (uppercase only, A-Z) and **nnnnn** is a five-digit number (possibly with leading zeros). The stock number of a product is assigned by eDEPOT when it receives the product for the first time.

Category

The category of the product (e.g., computers, hard disks, etc.) is stored. It is not necessary to maintain a hierarchy for product categories.

Manufacturer

This value is the name of the company that makes the product.

Model number

This is also a string representing the product type. The value is specific to each manufacturer.

Description

This entry contains detailed information about the product. A description consists of a list of *attribute-value* pairs. For example, for a computer system, the description may include:

processor speed: 2.5GHz, memory: 1GB, ...

The names of attributes are specific to the product.

Warranty

The number of months this product will be under warranty after the initial purchase.

Compatibility

This value is a list of products (specified by the manufacturer and the model number) that are compatible with this product, i.e., this product can replace any product in the list.

Price

The unit price of an item is a nonnegative real number.

2.2 Customers

The following information about customers should be stored.

Customer identifier

The customer identifier is a string assigned by the online store eMART to uniquely identify each customer.

Password

The password may be automatically generated by eMART or selected by the customer (e.g., during sign-up), and used to identify the customer when logging into eMART.

Name

A customer name may contain first, middle, and last names.

Email address

This is the email contact information for the customer.

Address

The customer's postal mailing address is also stored.

Status

The special status of the customer reflects the total amount of the last three purchases by the customer:

Gold : if the total purchase is $> \$500$.

Silver : if the total purchase is $> \$100$ but $\leq \$500$.

Green : if the total purchase is $> \$0$ but $\leq \$100$.

New : new customers who have not completed any purchases.

A customer with no prior purchases is a **new** customer. The status of a customer will be used for calculating discounts.

2.3 Customer Orders

eMART provides an interface for customers to search and purchase products (items). This resembles the functionality of many online retailers' web interfaces: in a single session, the customer can find an item, put in a shopping cart, and do the purchase. You may choose to save the shopping cart contents between sessions, but this is not required.

The online interface provides the following functionality (operations):

- *Searching* for item(s) by stock number, manufacturer, model number, category, description attribute and value, or *search* for compatible items of an item; a search condition can also be a combination of the above criteria.
- Operations on a shopping cart: *adding* an item, *deleting* an item, and *displaying* the current cart contents.
- *Checking out*: calculate the shopping cart total with the following rules for discount. A **new** customer or a **gold** customer will be given a 10% discount. A **silver** customer will be given a 5% discount. There will be no tax on purchases, but a 10% "shipping and handling" fee will be applied. If the shopping cart total (excluding shipping and handling) exceeds \$100 or the customer is **new**, the shipping and handling fee will be waived. The order will be confirmed with an order number and a list of items (quantity, price) in the order.

Note that the percentages and dollar amount in the discount rules and the shipping and handling policy may change at any time. It is thus necessary to store these rules in your database.

- *Displaying* a previous order for the given order number.
- *Re-run* of a previous order.

Typically, a customer searches for products, select one or more items from the search results to put into the shopping cart, and completes the purchase using the checking-out operation.

Questions: Should eDEPOT be checked for processing an order? And when if so?

2.4 Manager Interface

The manager interface is used by eMART employees to manage the store. It provides the following operations:

- Print monthly summary of sales: the amount (quantity, price) of sale per product, per category, and the customer who did the most purchase.
- Customer status adjustment according to the sales (can you automate this?) or manager's decision.
- Send an order to a manufacturer (shipment will go to eDEPOT directly).
- Change the price of an item.
- Delete all sales transactions that are no longer needed in computing customer status. All orders need to be stored unless they are explicitly deleted.

3 WAREHOUSE eDEPOT

The second component is the warehouse system eDEPOT that maintains a product inventory to support management of the collection of physical products at the warehouse.

3.1 Inventory

Specifically, the inventory maintains the following information for each product:

Stock number

When a product (or a shipping form containing the product) arrives at the warehouse for the first time, a unique stock number is created for the product.

Manufacturer and model number

They are the same as described in Section 2.1.

Quantity

The (nonnegative) quantity of the item in the warehouse clearly needs to be maintained.

Minimum and maximum stock levels

These numbers represent the lowest and highest quantity for this product eDEPOT will maintain. The warehouse will never store more than the maximum stock level. If a sale drops the quantity to below the minimum level, a new order might be generated (see the transaction part for details).

Location

This identified the location in the warehouse where the product is stored. We assume that one product is stored in one location. A location is identified by a letter (case insensitive) followed by a number (with no leading zeros).

Replenishment

The quantity of the item is currently being shipped to the warehouse.

3.2 Transactions

The following transactions must be supported by the warehouse system eDEPOT:

- Receive a shipping notice from a manufacturer; each notice includes a unique shipping notice identifier, the shipping company name, and a list of items (manufacturer, model number, quantity). In processing a shipping notice, the warehouse either finds its stock number, or assign a new one if it is a new product. The quantities in the shipping notice need to add to the items' replenishment entries to indicate future availability.
- Receive a shipment (always having with a prior shipping notice) from a shipping company. What processing needs to be done for the inventory?
- Check an item quantity in the inventory for a given stock number.
- Fill an order which has a unique order number, and a list of items sold (and their quantities). The purchase needs to adjust the inventory. If there are three or more items from the same manufacturer in the inventory that go below their respective stock level, send a replenishment order to the manufacturer. The replenishment order should include *all* products from the manufacturer that are below their respective maximum stock level.

4 OVERALL SYSTEM ARCHITECTURE AND DESIGN

Fig. 1 shows the main components of the systems you will develop.

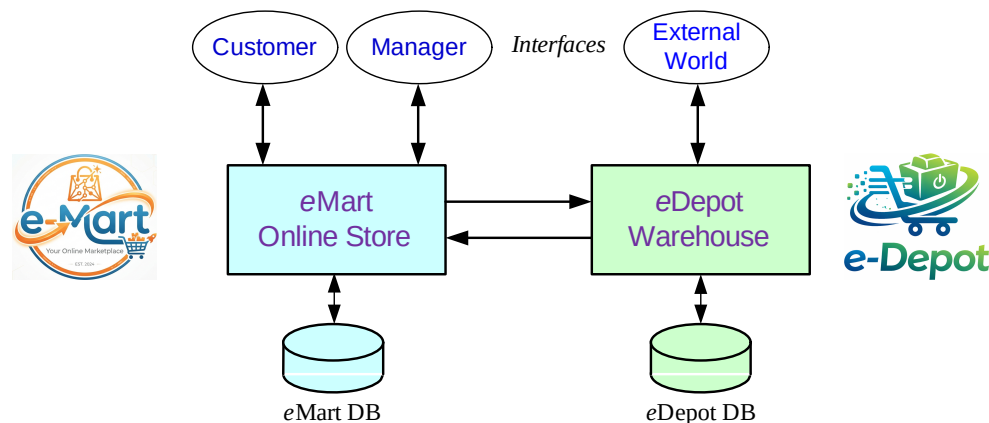


Figure 1: Components of Prototype Systems

- There are two software systems; the eMART and eDEPOT systems, which provides three interfaces: “Customer”, “Manager”, and “External World”.
- The two databases are maintained by eMART and eDEPOT, respectively. It is acceptable if you decide to manage the relations in the two databases using the same Oracle account, but it should be clear which tables belong to which database.
- The three interfaces for interactive users one for eMART customers, one for eMART managers, and one to simulate actions from the external world (vendors, etc.) to eDEPOT.

We now briefly discuss the components that you will design.

5 ONLINE STORE

The online store eMART will primary provide two functionalities:

1. It allows customers to purchase items available in the store through the customer interface. In particular, the interface will support all operations described in Section 2.3.
2. It allows the store manager to manage the store. This means that the manager interface will support the operations described in Section 2.4. Note that for the operation of sending an order to a manufacturer, you can simply “print” or display the order in some suitable form in your interface.

While you may have two programs for the two interfaces, conceptually, they constitute the eMART system.

6 WAREHOUSE

The second major component is the eDEPOT system. Roughly, the system manages the inventory in the warehouse. Its interface will support all operations described in Section 3.2.

As a simplification, eDEPOT will provide a simple user interface to handle all of its operations concerning the manufacturers, and devices such as RFID (Radio Frequency Identification) readers for receiving shipments, etc. The interface will simulate systems and agents that are interacting with eDEPOT system.

Clearly, eMART and eDEPOT will need to communicate with each other during operations, e.g., eMART checking availability of some items. Typically, this can be done in a few ways including, remote method invocations, socket programming, web services (RESTful or WSDL). However, you can choose your own way of communications. In particular, it will also be acceptable if your systems communicate “by hand”, e.g., printing out a checking inventory request, and manually checking inventory on eDEPOT and entering the result on eMART.

7 REQUIREMENTS

It’s desirable but *not required* that your systems have graphical user interface(s) for the customer and manager interfaces. Your interfaces need not be accessible from a web browser.

You must store all data of your systems in the databases managed by the Oracle DBMS. This means that when your systems are not running, all data are in the databases and nothing would be stored anywhere else including files. (During the demo, your systems may be asked to shut down and restart.) All previous actions done on your system must be remembered. (Again, you should not use files.) Your system should be implemented in Java using JDBC to connect to your database managed by the Oracle DBMS.

7.1 Project Team

The course project (system design and implementation) is to be completed by each team consisting of 1-2 students. Although both the difficulty and amount of work are suitable for either one or two students, it is strongly encouraged that you work with another student in a team.

Within each team, one student will be primarily responsible for developing eMART (the eMART developer), and the other for eDEPOT (the eDEPOT developer). However, we suggest the following:

- The eMART developer should use the database schema designed by the eDEPOT developer; similarly, the eDEPOT developer should use the database schema designed by the eMART developer. Database schemas may be changed, but every change must be approved by the original designer. Also, all schema changes (to the version submitted in the preliminary report) must be documented in the final project report.
- Every one in the team is expected to know all details of the implementation up to the level of being able to answer questions concerning design decisions.

7.2 Data Design Report

Your first task for the project is data design. You are asked to develop the database schemas, one for eMART and one for eDEPOT. Although the project is done by groups of two students, you must complete this part of the project alone. Your report should contain the following:

1. Draw ER diagrams for the schemas.
2. Translate your ER diagrams into relations.
3. For each operation/transaction that will be provided by eMART and eDEPOT, list the table(s) and column(s) to be read, and the table(s) and column(s) to be inserted/updated.
4. Identify as many integrity constraints for the relations as you can. Will some of the operations violate a constraint you listed?
5. Write the SQL CREATE TABLE statements for the relations and include all primary/candidate/foreign key constraints you have listed in Item 4.

You may consider using drawing tools such as <https://drive.draw.io/> or <http://www.gliffy.com> or <http://ermaster.sourceforge.net/>. You will turn in an electronic document (PDF or word document) on Gradescope on or before Wednesday, May 13.

7.3 Project Report

At the completion of your project, each team must submit a project report. The report has to address the issues considered in making major design decisions. The report should be based on and a revision of your data design report, and addresses the following issues:

- Identify who's responsible for developing eMART and eDEPOT and list the task divisions.
- Show the final database schemas and all integrity constraints. Discuss each change you have made to the database schema after the early report.
- Describe briefly how you deal with a violation of each of the identified integrity constraints.
- Show a picture of each of the three interfaces, and a brief description about them.

In addition, a (tar/zip) archive of all source code and supporting files should also be submitted (separately from the final report).

7.4 Demo

At the end of your project, a 30-minute demonstration of your systems operations has to be presented to the teaching staff of this course. The project deadline is at your scheduled demo. *Regular* demos will be arranged during the last week of instruction (the 10th week, i.e., June 1 to June 5). As an option, *early* demos can be arranged during the 9th week (i.e., May 26–29). Each early demo will be given a 10% extra credit of the earned project score. You have to choose in advance between regular and early demos; no teams will be allowed to have two or more demos. We will provide a sample dataset (including customers, accounts, and a list of transactions). This data should be entered into your database *before* your demo. The details about the demos will be announced at a later time.

7.5 What To Turn In at Your Demo

1. A final project report, and
2. A (tar/zip) archive of all Java source code and related files.